

Distributed Optimization Mechanism for Increasing Renewables Penetration Rate in Energy Communities

Adrian-Marian Schiop
Computer Science
Department
Technical University of Cluj-
Napoca
Cluj-Napoca, Romania
Schiop.Ad.Adrian@student.u
tcluj.ro

Dan Mitrea
Computer Science
Department
Technical University of Cluj-
Napoca
Cluj-Napoca, Romania
dan.mitrea@cs.utcluj.ro

Tudor Cioara
Computer Science
Department
Technical University of Cluj-
Napoca Cluj-Napoca,
Romania
tudor.cioara@cs.utcluj.ro

Abstract—Renewable Energy Communities (RECs) empower prosumers to integrate renewable energy sources at the network edge. However, the intermittent generation of energy can produce sudden spikes that risk line overloads and voltage excursions. Optimizers can improve local grid management, and Particle Swarm Optimization (PSO) is a popular approach for its ability to handle non-linear dynamics, however it struggles with high-dimensional contexts. In this paper, we propose a distributed, fully on-chain Particle Swarm Optimization (PSO) algorithm, for transparent optimization within RECs. Each prosumer deploys a Node smart contract that runs a local PSO instance, enhanced for the energy domain, which considers individual energy profiles and custom penalties to compute a local cost. Then, a lightweight Global Coordinator contract aggregates each node's best positions, weighted by their flexibility band, to compute a system wide plan which every prosumers tries to follow in an iterative process. We evaluate the solution using real consumption data in scenarios with up to 500 prosumers, showing that the solution converges and reduces the cost of each prosumer, while increasing the renewable energy usage in the community.

Keywords—blockchain, smart contract, particle swarm optimization (PSO), smart grid, energy optimization

I. INTRODUCTION

The global drive towards net-zero emissions has led to a rapidly increasing penetration of distributed renewable energy sources (RES) at the edge of the network. Smart Grids have emerged as key enablers for integrating these intermittent resources, empowering prosumers and enabling real-time adaptability of supply and demand [1]. Renewable energy communities (REC) are contributing to the decentralization of energy in local communities, enabling prosumers to collaborate in how they produce and consume energy, thus enhancing local resilience and reducing the dependence on the main grid, while also opening the door to peer-to-peer energy exchange [2].

Despite their promising advantages, RECs introduce significant operational challenges: renewable energy generation is intermittent and variable, and can lead to sudden spikes in local production, that can risk overloading distribution lines or causing voltage excursions. Classical, centralized optimization approaches can balance supply and demand and mitigate these effects. However, centralized approaches suffer from single points of failure, lack of transparency for participants, and data tampering vulnerability [5]. Decentralization emerges as a promising alternative, due to blockchain technology's inherent properties of

immutability, distributed consensus and transparency. It can provide a robust infrastructure for smart grid optimization. Blockchain can handle interactions between peers without the need of a third-party entity via smart contracts [6]. However, capturing both consumption flexibility and network constraints is difficult, and can lead to inefficiencies or grid instability.

Since energy management in smart grids represents non-linear dynamics, a growing number of solutions in the literature are applying heuristic and metaheuristic methods, such as Genetic Algorithms or Bio-inspired approaches, due to their ability to accurately approximate optimal solutions and rapid convergence [3][4]. Particle Swarm Optimization (PSO) has been widely studied and used as a bio-inspired optimization metaheuristic, because it offers rapid convergence and low computational complexity. However, in high-dimensional energy contexts, it showed slow convergence towards global optimum and might struggle with complex power network constraints. These issues are even more pressing when implementing PSO on-chain, in smart contracts. Moreover, other on-chain implementations rely on centralized coordinators that not only aggregates global system information but also holds complete knowledge of the system state [5]. In parallel, blockchain-based solutions have been proposed to secure decentralized energy transactions between prosumers, and enforce agreements through smart contracts, without intermediaries [7][8]. However, there are a few existing solutions that focus on on-chain optimization for energy management in smart grids, which enforce network feasibility under intermittent renewable conditions.

To address the open challenges in the state of the art, we propose a distributed, on-chain Particle Swarm Optimization (PSO) solution that optimizes local energy communities to utilize more renewable, local energy, by adjusting energy flexibility. The solution is fully implemented in smart contracts on the Ethereum platform. We enhance the standard PSO algorithm for the energy domain. Each prosumer is represented by a smart contract which runs a PSO instance, modeling individual prosumer energy profiles and integrating penalties for energy flexibility violation and overconsumption. A Global Coordinator smart contract aggregates only the best positions and computes a system wide plan that follows network and flexibility constraints. Each PSO instance tries to follow this plan by also applying their individual constraints.

This article has the following contributions:

- We adapt the classical PSO algorithm to the energy domain, introducing energy profiles modeling and custom penalties for flexibility violations and overconsumption.
- We design the PSO optimization mechanism fully on-chain, in smart contracts on the Ethereum network, thus providing auditable, immutable optimization records on the blockchain. Although we leverage a Global Coordinator that makes the architecture distributed and not fully decentralized, we improve the state-of-the-art by not storing the state of the entire energy network, just relevant parameters about the nodes in relation with the PSO algorithm state.
- We evaluate the solution with real data, in scenarios with up to 500 prosumers, evaluating the correctness and convergence of the PSO optimization, as well as operational costs in terms of gas and execution time, and finally, scalability.

The rest of the paper is structured as follows: Section II presents the existing state of the art in smart grid energy consumption optimization, Section III shows how we model and implement the adapted PSO mechanism for the energy grid. Section IV presents evaluation results and Section V concludes the paper.

II. RELATED WORK

Smart grids and renewable energy sources have become increasingly popular in recent years. Due to their mass adoption, they raise a new critical challenge, that smart grids are of bidirectional, variable and intermittent nature, when handling electrical energy. In the traditional conventional grids, energy is transmitted from the source to the end user. However, in smart grids, energy can flow both ways, allowing not only for electricity to be delivered to the user, but also for excess energy generated by prosumers to be fed back into the grid.

To meet global climate goals such as Paris Agreement, renewable energy (RE) must form a substantial portion of future electricity supply [9]. The concept of Renewable Energy Clusters presented in [10] addresses these requirements by integrating smart grid technologies, distributed energy resources (DERs). RE clusters rely on digital solutions such as smart meters, peer-to-peer (P2P) trading, virtual power plants (VPP). In addition to smart grid coordination mechanisms, VPP play an important part in optimizing decentralized energy production as described in [11] a VPP aggregates DERs into a single controllable entity. A VPP relies on cloud-based software platforms to monitor, forecast and optimize the dispatch of energy in real time.

Blockchain technology has been proven to be very effective in decentralizing energy systems [12]. Blockchain is a distributed database of transactions records validated and maintained by a network of computers. By removing centralized authorities, blockchain can effectively reduce costs and improve both control and competition of small size prosumers. Smart contracts [14] can automatically enforce the terms of agreements between two or more entities; thus they can automate the delivery of energy services and enforce decentralized control of energy assets. However, security is a

critical concern in decentralized smart grids due to their increasing number of interconnected energy devices. As outlined in [13] blockchain can effectively prevent cyber threats such as data tampering, denial-of-service (DoS) attacks, and unauthorized access, thanks to its consensus protocols and distributing ledger design.

One of the main challenges when integrating renewable energy sources into local energy grids is that they might produce sudden spikes in energy production at certain times of the day, potentially damaging the energy grid [18]. DR methods serve in reducing total power consumption, not only at the consumer level, but also in terms of transmission and distribution losses, generating mutual benefits for both parts. Moreover, DR methods play an important role in reducing the need to activate polluting power plants by decreasing peak demand. Moreover, peak demand can be reduced by analyzing historical and real-time energy consumption data to identify usage patterns and forecast high-demand periods [19]. Based on the data, the system delays or reschedules flexible loads and guides consumers to shift their usage away from peak hours. By shifting their usage of the grid to avoid peak congestion, prosumers receive monetary compensation proportional to the degree of flexibility they provide [20].

Smart grid management optimization can be applied to individual prosumers, or to the entire community, considering energy exchanges between prosumers, or between prosumers and the energy grid. In [28], the entire energy community is considered, and optimization is applied to energy sharing and balance. Another solution tries to also integrate both user comfort level and renewable energy consumption rate [27]. In this article, a centralized version of the PSO algorithm is executed, with the objective function considering the reduction of daily energy costs, composed of renewable generation costs and fuel cells. It considers the constraints of power conservation, battery operation and user comfort. Results show that if considering user comfort level, the renewable usage rate decreases, but might offer higher adoption rates. Two tightly coupled optimization problems, in the context of RECs, namely participant selection that minimize energy generation surplus, and energy allocation to balance economic payback and production surplus, are tackled in [29]. For selection, a Genetic Algorithm is applied, with the fitness aiming to minimize total production surplus, which generates a subset of prosumers that can absorb the maximum solar output. Then, for the allocation, a multi-objective genetic approach is employed, aiming to increase economic payback and lower solar excess for a subset of prosumers, having given their consumption and production capabilities. Another approach [30] aims to optimize net energy balance per prosumer, by minimizing grid imports and PV surplus, total economic savings and equity and fairness constraints for prosumers. A Genetic algorithm is employed for economic allocation, where the fitness is given by total hourly savings. Then, the Water-Filling algorithm is used to pour available energy into the lowest demands first, allocating iteratively until resources are exhausted.

Since blockchain can greatly improve security, immutability and decentralized trust, DERs can be coordinated in a decentralized manner through smart contracts [15]. A decentralized coordinator can replace traditional, centralized aggregators. Each DER performs local optimization, while a smart contract deployed on blockchain serves as the global aggregator in an Alternating Direction

Method of Multipliers (ADMM) scheme. This approach ensures feasibility and eliminates the risk of market manipulation by centralized aggregators and enables fair payments directly between peers. Through blockchain, the Optimal Power Flow (OPF) problem can be solved through ADMM [16], where each node is treated like an autonomous agent that performs local optimization based on energy consumption, production and operational constraints. Agents exchange encrypted data, preserving privacy. Through smart contracts, the convergence conditions are checked and then, the aggregated values are broadcasted for the next iteration.

Prosumers and local energy communities play a pivotal role in integrating renewable energy sources into traditional energy systems without expensive infrastructure upgrades [20]. Moreover, these traditional infrastructures rely on long-term, fixed contracts, having drawbacks for prosumers with complementary profiles, leading to pricing volatility. A decentralized P2P energy platform is proposed, using Hawk-Dove evolutionary game theory for reducing transmission losses and improving grid balance. A multi-agent system that integrates blockchain and metaheuristic algorithms is proposed in [21]. Flexibility constraints are mixed with consumer preferences to ensure both economic and operational optimization, proving the potential of metaheuristic solutions in enhancing decision-making and energy flow coordination. A decentralized genetic algorithm is proposed [22], where IoT devices integrated with the smart grid, perform local optimizations. Through blockchain and smart contracts, they share their best solutions to a global contract. This approach allows the population of solutions to collectively evolve, while maintaining optimal power flow across the network.

Another promising bio-inspired metaheuristic, drawing inspiration from the natural behavior of birds searching for the most suitable place in a flock, is the Particle Swarm Optimization (PSO) algorithm [23]. While this algorithm has natural advantages, such that it does not involve selection or mutations, it can be applied in various engineering applications and has few parameters to tune [25], it also has some disadvantages, such as a lack of dimensionality, making it unsuitable for problems of non-coordinate systems such as the energy field where it converges very slowly towards the global optimum. Due to this, many solutions try to adapt PSO for specific cases, either modifying PSO parameters or combining PSO with other meta-heuristics [24].

III. OPTIMIZATION USING PARTICLE SWARM ALGORITHM

A. Mathematical Model

We propose a solution that optimizes energy consumption in a smart grid, leveraging an enhanced Particle Swarm Optimization (PSO) algorithm. We choose this method for its convergence speed and conceptual simplicity in handling continuous optimization problems with limited computational overhead. We introduce several improvements to the classical PSO, such as custom threshold variables that penalize overconsumption, flexibility violations and deviation from global targets, as well as velocity updates bounded by personalized flexibility constraints.

The velocity update incorporates inertia from previous behavior, personal memory via the best individual plan and social influence through the globally optimal plan obtained from global contract. Velocity updates are bounded by the node's own flexibility margins, ensuring that the resulting

consumption remains within the physical capabilities of the prosumer.

We compute the velocity the following way:

$$v_i^{t+1} = \frac{1}{100} * (w * v_i^t + c_1 * r_1 * (pBest_i - x_i^t) + c_2 * r_2 * (gBest_i - x_i^t) + \epsilon_i), \text{ where } \epsilon_i \in [-50, 50] \text{ \& } i \in \{0, 24\} \quad (1)$$

where w represents the inertia factor, which controls the influence of previous velocity on the current update. c_1 is a cognitive factor reflecting the node's individual learning based on its best solution discovered, c_2 is a social component that reflects the best solution in the network, r_1 and r_2 are random variables for ensuring variability and exploration capability within the solution space. As an enhancement brought to the classical PSO algorithm we introduced the perturbation factor ϵ_i to improve diversity of the swarm and to avoid premature convergence. The bounds -50 and +50 were empirically chosen after iterative simulations to maintain a balance between local exploration and overall network stability.

The position update is the result of adding the current velocity to the previous position, reflecting the change in the node's energy consumption:

$$x_i^{t+1} = \min(\max(x_i^t + v_i^{t+1}, pBest_i - 2 * flexibilityBelow_i), pBest_i + 2 * flexibilityAbove_i) \quad (2)$$

where *flexibilityBelow* represents the minimum recorded consumption per hour over the past 7 days and *flexibilityAbove* is the maximum recorded consumption per hour over the same period.

The fitness function evaluates the quality of each energy consumption plan proposed by each prosumer, transforming the 24 consumption values into a numerical score. In our implementation we incorporate specific constraints and performance metrics to evaluate not only the raw consumption cost (C_i) but also introduce penalties for operational inefficiencies as well as incentives for alignment with global plan. (P_i, A_i) The mathematical model for the fitness function is computed as the sum of all costs and penalties for each hour of the day (3).

$$fitness = \sum_{i=0}^{23} [C_i + P_i + A_i] \quad (3)$$

Where C_i denotes the cost of energy purchased from the grid by the node at hour i . The cost is calculated by multiplying the consumption value with energy tariff. P_i is the penalty for overconsumption, instability and unused renewable energy calculated in (4) A_i represents the global adjustment which reflects how well the node follows the global plan (5).

$$P_i = \alpha * \max(0, pos[i] - capacity[i]) + \beta * \max(0, pos[i] - personalBest[i] - (flexAbove_i + flexBelow_i)) + \gamma * \max(0, R_i - pos[i]), \quad (4)$$

where α, β, γ represent the custom threshold coefficients for penalizing inefficient behavior and R_i represents the renewable energy that is available at hour i . α penalizes overconsumption beyond the node's capacity, β discourages large deviations from node's own best historical plan. If the proposed consumption plan deviates from the personal best

above the allowed flexibility margin a static penalty is applied. γ introduces a cost related to unused renewable energy.

$$A_i = \begin{cases} PENALTY_{GLOBAL} * (|d_i| - \theta_i), & \text{if } |d_i| > \theta_i, d_i > 0 \\ -REDEEM_{GLOBAL} * (|d_i| - \theta_i), & \text{if } |d_i| > \theta_i, d_i < 0 \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

where d_i is the difference between the node's consumption and the global plan at hour i and θ_i represents the tolerance threshold for deviation from the global plan defined as 10% of the global plan value. If the deviation remains within the allowed tolerance no adjustment is made. However, when the node significantly deviates from the global objective an adjustment is triggered.

When all the prosumers send their data to the global aggregator, the smart contract runs a weighted average based on the flexibility of each prosumer. The aggregated value for each hour is calculated in (6).

$$globalPlan = \frac{\sum_{j=1}^N flexWeight_j[i] * bestPosition_j[i]}{\sum_{j=1}^N flexWeight_j[i]} \quad (6)$$

The fitness function acts like a loss function. The PSO algorithm aims to minimize this value by making successive adjustments to the consumption and the velocity of each particle, taking into consideration their flexibility.

B. Algorithm implementation

We have designed and implemented the adapted PSO algorithm entirely in smart contracts on an Ethereum blockchain infrastructure. We model the system with two entity types, multiple Node smart contracts and one Global Coordinator smart contract. Each prosumer in the community will be represented by a Node smart contract.

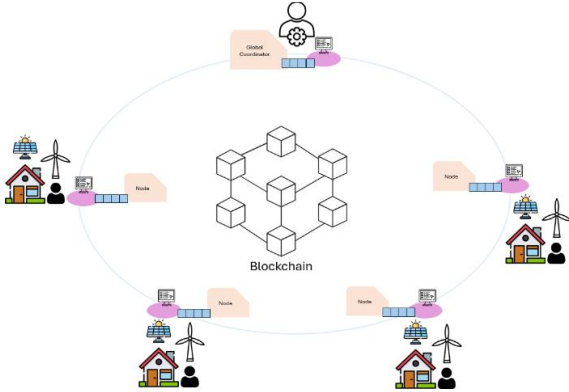


Fig. 1. Smart Grid Optimization Using On-Chain Particle Swarm

The global coordinator receives updates from each node and then computes a global optimum plan, allowing nodes to adapt their local solution towards the global plan iteratively. A general smart contract overview is presented in Fig. 1.

```
pragma solidity ^0.8.0;
struct Node {
    uint256[] position;
    uint256[] velocity;
    uint256[] tariff;
    uint256[] capacity;
    uint256[] renewableEnergy;
    uint256[] batteryCapacity;
    uint256[] batteryCharge;
    uint256[] flexibleLoad;
    uint256[] flexibilityAbove;
    uint256[] flexibilityBelow;
}
```

Fig. 2. Core Parameters Defining a Prosumer Node in Smart Grid

Each node smart contract begins by initializing its internal state. The position vector is set to represent the initial 24-hour consumption plan, while the velocity vector governs the magnitude and direction of change in the consumption profile. *Tariff* represents the cost of energy in an hour, *capacity* represents the maximum allowable energy drawn from the grid for a node. The *renewableEnergy* variable represents the amount of self-generated green energy available in an hour. The energy profile is represented in the last three variables. These parameters define the node's permissible range of deviation to achieve an optimal global plan.

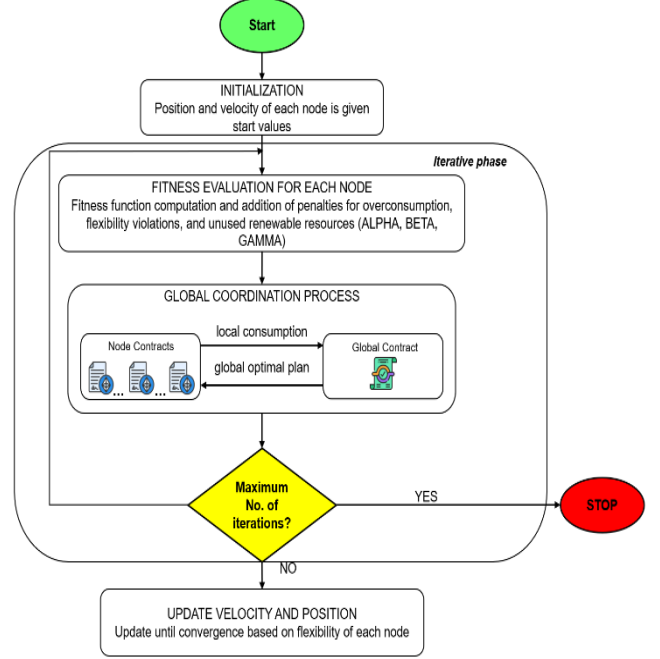


Fig. 3. Flowchart of PSO

Fig. 3 presents the general process of the adapted PSO algorithm we have developed. We begin by initializing the position and velocity vectors for each node, then each Node computes their local fitness for their proposed plan through an objective function described in Relation 3. This fitness score is computed on-chain for the entire period of 24 hours of the day. After computing the fitness score, each Node contract communicates their preferred plan to the Global Coordinator, which computes an aggregation of the best plans for the entire community. After that, the global coordinator sends this plan to each Node, which will try to follow it within the bounds of their flexibilities. This process is repeated iteratively, with the global coordinator updating the best overall plan based on the updates of all the nodes from each iteration. Finally, after the iterations end, each Node will have the best possible local plan, with the Global Coordinator having the latest and most cost-effective solution for the community.

The process in which each Node smart contract sends its current best solution, stored as a state variable, to the global coordinator, including the associated cost and flexibility weight vector, is presented in Fig. 4. This vector is derived from the node's declared upper and lower flexibility margins and it expresses the ease with which a node can adapt its consumption during each hour of the day. This parameter plays a key role in the aggregation of local consumption plans to form a coherent global strategy.

```

1 function updateBestPositions() public {
2     int currentScore = objectiveFunction(position);
3     if (currentScore < personalBestScore) {
4         personalBestScore = currentScore;
5         personalBestPosition = position;
6         delete frozenRenewableGeneration;
7         delete frozenBatteryCharge;
8         for (uint i = 0; i <
9             renewableGeneration.length; i++) {
10
11 frozenRenewableGeneration.push(renewableGeneration[i]);
12         frozenBatteryCharge.push(batteryCharge[i]);
13     }
14 }
15 uint[] memory flexWeights =
16 calculateFlexibilityWeight();
17 globalContract.updateNodeResult(position,
18 currentScore, flexWeights);
19 }

```

Fig. 4. Communication of best position to the global coordinator

This functionality decides if the current fitness, obtained after adapting the previous best solution to the global plan from the coordinator, is better than the previous best solution – lines 2-3. If so, we update the best score and its position – lines 4-5, while also storing a snapshot of the critical parameters that tell under which conditions that score was found, namely the renewable generation capacity, as well as the battery charge – lines 8-14. These will be used later to recompute the score under the exact same conditions. Finally, we compute the *flexibilityWeights* as the average of the *flexAbove* and *flexBelow* for each hour – lines 15-16. This score expresses the ease with which a Node can adapt its consumption during each hour of the day and plays an important role in the aggregation of local consumption plans to form a coherent global strategy. Then it sends the most relevant arguments to the global contract which updates the information related to this Node – lines 17-18.

```

1 contract GlobalCoordinator {
2
3     struct NodeResult {
4         int[] bestPosition;
5         int bestScore;
6         uint[] flexibilityWeight;
7         bool exists;
8     }
9
10    mapping(address => NodeResult) public nodeResults;
11    address[] public nodeAddresses;
12    mapping(uint => int) public globalOptimalPlan;
13    uint public numHours;
14
15    int public bestGlobalCost = type(int).max;
16    int[] public bestGlobalPlan;
17 }

```

Fig. 5. Global Coordinator smart contract state

Fig. 5 presents the state variables stored in the Global Coordinator smart contract. For each node's blockchain address we store a struct that keeps its best positions, best score and flexibility weights that will give it a higher score in computing the global plan – lines 3-8. The *NodeResult* for each Node will be updated when the Node communicates with the global coordinator through the mechanism presented in Fig. 4. After each Node sends their new best score, if there is one, the Global Coordinator will compute the latest best global plan for this iteration, as presented in Fig. 6.

```

1 function computeGlobalOptimalPlan() public {
2     for (uint i = 0; i < numHours; i++) {
3         int weightedSum = 0;
4         uint totalWeight = 0;
5         for (uint j = 0; j < nodeAddresses.length;
6             j++) {
7             if
8 (nodeResults[nodeAddresses[j]].exists) {

```

```

9         uint weight =
10 nodeResults[nodeAddresses[j]].flexibilityWeight[i];
11         weightedSum +=
12             int(weight) *
13 nodeResults[nodeAddresses[j]].bestPosition[i];
14         totalWeight += weight;
15     }
16 }
17 if (totalWeight > 0) {
18     globalOptimalPlan[i] = weightedSum /
19 int(totalWeight);
20 } else {
21     globalOptimalPlan[i] = DEFAULT_VALUE;
22 }
23 }
24 lastUpdatedTimestamp = block.timestamp;
25 int currentNetworkCost = 0;
26 for (uint j = 0; j < nodeAddresses.length; j++) {
27 {
28     if (nodeResults[nodeAddresses[j]].exists) {
29         currentNetworkCost +=
30 nodeResults[nodeAddresses[j]].bestScore;
31     }
32 }
33 if (bestGlobalPlan.length == 0 ||
34 currentNetworkCost < bestGlobalCost) {
35     bestGlobalCost = currentNetworkCost;
36     if (bestGlobalPlan.length != numHours) {
37         bestGlobalPlan = new int[](numHours);
38     }
39     for (uint i = 0; i < numHours; i++) {
40         bestGlobalPlan[i] =
41 globalOptimalPlan[i];
42     }
43 }
44 for (uint j = 0; j < nodeAddresses.length; j++) {
45     Node(nodeAddresses[j]).
46         updateVelocityAndPosition(bestGlobalPlan);
47 }
48 }

```

Fig. 6. Computation of the best plan in each iteration

For each hour of the day, the solution iterates over all the registered nodes – lines 2-5, and aggregates the optimal consumption of each node (*bestPosition*) using a weighted average, where the weight is determined by the flexibility score computes locally, which represents the average between *flexAbove* and *flexBelow* – lines 9-14. This flexibility score shows how flexible the node is and gives it a higher weight. Thus, the global ideal plan is this weighted average score – lines 17-23. The solution also computes the sum of individual node scores as the *currentNetworkCost* – lines 26-32. If the global plan computed in the beginning of the method, which represents the updated global cost, is lower than both the previous value and the current network cost, we update it as the local best global plan – lines 33-43, then send it to each Node – lines 44-47. Leveraging the current global optimum, each node updates their velocity and position within the PSO, using Relation (1) and Relation (2), and using the global optimum for the argument *gBest*.

IV. EVALUATION RESULTS

To test the effectiveness of the adapted, distributed PSO for local, energy communities cost optimization, we have constructed a realistic scenario with data extracted from the University of California archive, which represents the electricity usage of 370 consumers, recorded at 15-minute intervals in 2011-2014 [26]. A resampling operation was applied to aggregate the data on an hourly basis, allowing for the calculation of hourly values. Since these values represent monitored energy consumption values, we have adapted them to obtain the energy flexibility profiles of each consumer. The flexibility profiles were generated by using historical consumption data from the previous 7 days to compute an average (*flexibleLoad*), representing the baseline, a

maximum (*flexibilityAbove*) and a minimum (*flexibilityBelow*) value.

However, data regarding price of energy, capacity of nodes, and battery capacity values are generated in a synthetic way. The price of energy was generated by sampling random values between 15 and 45 for each hour. The capacity of a node was computed as 120% to 150% of the corresponding hourly consumption. Renewable generation vector was calculated as up to 40% of each hour's consumption value, reflecting variable green energy inputs from local, community sources, like photovoltaic panels. Each node was also equipped with a synthetic battery system, with battery capacity defined as 50% to 100% of its corresponding capacity and battery charge randomly selected within that range. We have tweaked the c_1 parameter, which in the classical PSO algorithm was 150, to 200, to obtain a more sustained convergence. Finally, we have used the following values $\alpha = 75$, $\beta = 43$, $\gamma = 18$ such that the algorithm converges in less than 20 iterations, no matter in what time of the day we execute the algorithm – otherwise the random factors, computed pseudo-randomly based on timestamp, could influence the convergence speed.

We have chosen a random selection of 28 consumers out of the 370 to model an energy community and test our solution. The scenario that we constructed uses the data adapted from [26] to generate consumption values over 24 hours of a day, ranging from 0 to 121 kWh, value appropriate for buildings from small homes to even neighborhood supermarkets. For example, the energy profile of Consumer 1 is presented in Fig. 7.

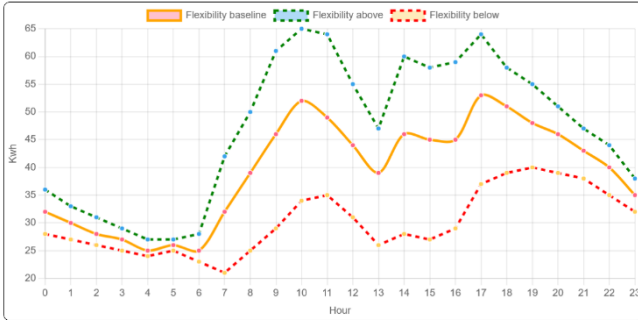


Fig. 7. Flexibility curves for a prosumer

We begin by analyzing the impact of the optimization process on the individual consumption profile of a prosumer in Fig. 8, by comparing their consumption over 24 hours of a day, which initially was completely from the grid, without using any renewable energy. However, after the optimization process, we can see a new consumption pattern, where in most hours, it was lower than the initial consumption values, but for a few hours, it became larger, becoming more tailored to the community global optimum. We can also see that for all hours, the energy used from the grid lowers, and for each hour, they start using renewable energy, either from renewable sources directly or stored in batteries. In conclusion, the final, optimized plan is better tailored to the community objectives, while simultaneously reducing the energy consumed from the grid, increasing the renewable use, all while following the bounds of flexibility of the prosumer presented in Fig. 7.

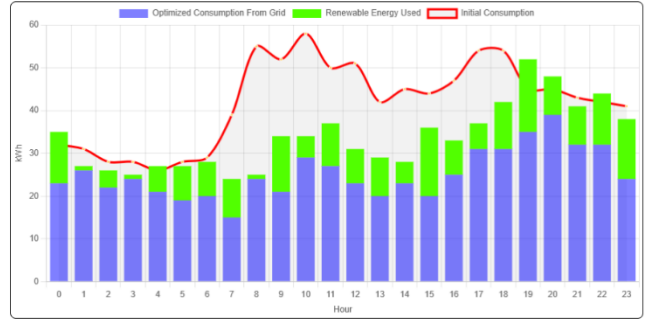


Fig. 8. Prosumer optimization results

Fig 9 shows the impact of optimization on the entire energy community. We can see the total energy consumption for a day, both from the main grid and from renewable sources, for the entire scenario of 28 prosumers, and how it changes as iterations advance. We can see that after 4-5 iterations, the solution converges to the final values, which remain constant across 20 iterations of the enhanced PSO algorithm. We can also see that the consumption from the main grid decreases, while the renewable usage increases. Moreover, the grid consumption decreases more than the renewable increase, signaling a total energy consumption lower than the initial one, indicating a lower, optimized consumption of the energy community overall.

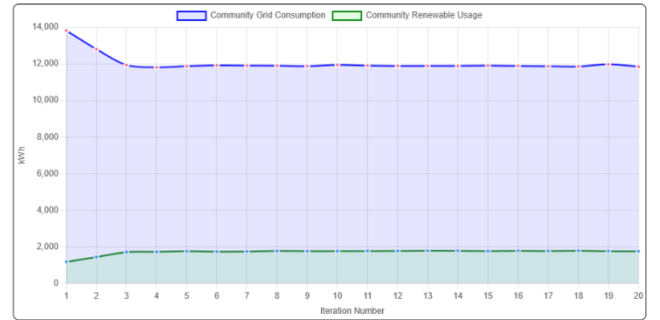


Fig. 9. Energy community consumption optimization

Moving further, we aim to analyze the operational costs of this solution. We conducted tests to measure both efficiency and costs in the context of using the Ethereum network. We will measure the cost in Gwei, which is a denomination of the cryptocurrency Ethereum, namely one-billionth of one Eth, and is the most used ether unit when referring to Ethereum transactions. Overall, the application operates efficiently, providing an adequate response time for all operations. The cost of executing the global plan update is 8 times more expensive than for the node to update their velocity, and the execution time of the coordinator is 4 times larger than that of updating each node. The results can be seen in Table 1.

TABLE I. COST AND TIME OVERHEAD FOR MAIN USE CASE

Use Case	Cost (Gwei)	Time (s)
Global plan aggregator	4744227	32.1
Node position and velocity update	625411	7.8

Furthermore, to assess scalability, we simulated the behavior of the network with an increasing number of prosumers. To conduct this scalability test we use the data from [26] to generate similar energy profiles to the initial

scenario, but for 500 users. Even though [26] provides consumption profiles for 350 unique users. To extend the dataset to 500 users, we duplicated a subset of the existing profiles. While this approach introduces some data redundancy, it allows us to evaluate the system's performance under large-scale conditions and to examine how execution time and cost evolve with network size. Fig 10 and Fig 11 demonstrate that both gas cost and execution time scale exponentially with the number of prosumers in the network. This phenomenon occurs because the PSO algorithm is fully executed on chain, and all the optimization calculations are performed directly within the smart contracts. These results show that the application can still function properly in communities with 500 participants, with gas costs and time durations that are acceptable in this case, where the optimization is executed once per day. However, if the number of participants continues to increase, it might pose difficulties to integrate it in the community.

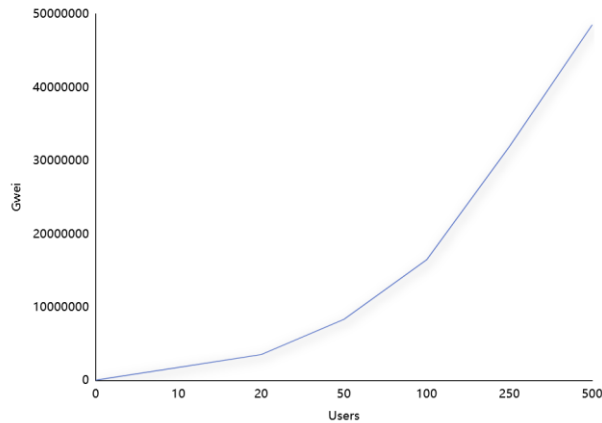


Fig. 10. Cost evolution with the number of users

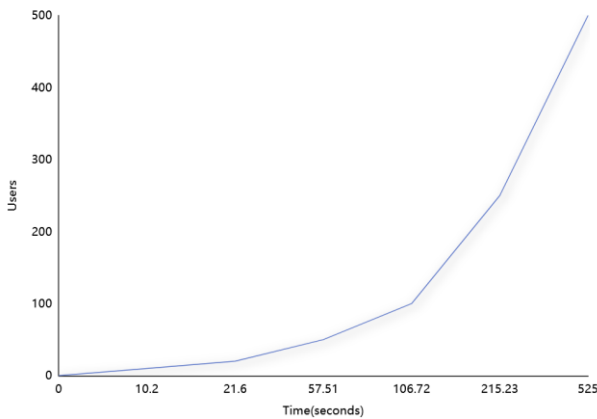


Fig. 11. Time overhead evolution with number of users

V. CONCLUSIONS

In this paper we described an enhanced, distributed PSO algorithm, implemented in smart contracts on the Ethereum network, which could be applied to a smart grid in a local community. We enhanced the classical PSO algorithm, which is not suited for the energy domain, by introducing penalties for multiple custom conditions: exceeding prosumers' flexibilities, deviating from the global optimal plan, and underutilizing renewable energy. The penalties were integrated directly into the fitness function of each node allowing the optimization to reflect both local and system-

wide constraints. Moreover, we perform velocity updates for each node, based on the owner prosumer's flexibility constraints. Each prosumer was represented by a smart contract deployed on Ethereum blockchain executing its own PSO instance independently. Smart contracts exchanged optimization data with a central aggregator SC in a trusted and transparent manner. The global plan was computed as a weighted aggregation of the best solutions submitted by the nodes.

The evaluation results show that our PSO algorithm implementation can reduce energy consumption peaks, improve load distribution, and increase renewable energy usage all while maintaining the autonomy of each prosumer.

ACKNOWLEDGMENT

This work was supported by the project "Romanian Hub for Artificial Intelligence-HRIA", Smart Growth, Digitization and Financial Instruments Program, MySMIS no. 334906.

REFERENCES

- [1] Ahat, Murat, et al. "Smart grid and optimization." *American Journal of Operations Research* 3.1 (2013): 196-206.
- [2] Ahmed, Shoaib, Amjad Ali, and Antonio D'angola. "A review of renewable energy communities: concepts, scope, progress, challenges, and recommendations." *Sustainability* 16.5 (2024): 1749.
- [3] Torkan, Ramin, Adrian Ilinca, and Milad Ghorbanzadeh. "A genetic algorithm optimization approach for smart energy management of microgrids." *Renewable Energy* 197 (2022): 852-863.
- [4] Wang, Dongshu, Dapei Tan, and Lei Liu. "Particle swarm optimization algorithm: an overview." *Soft computing* 22.2 (2018): 387-408.
- [5] Mourtzis, Dimitris, and John Angelopoulos. "Reactive power optimization based on the application of an improved particle swarm optimization algorithm." *Machines* 11.7 (2023): 724.
- [6] Muzumdar, Ajit, Chirag Modi, and C. Vyjayanthi. "A trustworthy and incentivized smart grid energy trading framework using distributed ledger and smart contracts." *Journal of network and computer applications* 183 (2021): 103074.
- [7] Aloqaily, Moayad, et al. "An energy trade framework using smart contracts: Overview and challenges." *IEEE Network* 34.4 (2020): 119-125.
- [8] Guo, Y., Z. Wan, and X. Cheng. "When blockchain meets smart grids: a comprehensive survey. High-Confidence Comp 2 (2): 100059." 2022.
- [9] Delbeke, Jos, et al. "The paris agreement." *Towards a climate-neutral Europe*. Routledge, 2019. 24-45.
- [10] Lowitzsch, Jens, Christina E. Hoicka, and Felicia J. van Tulder. "Renewable energy communities under the 2019 European Clean Energy Package-Governance model for the energy clusters of the future?." *Renewable and Sustainable Energy Reviews* 122 (2020): 109489.
- [11] Bedi, Aakanksha, et al. "Empowering net zero energy grids: a comprehensive review of virtual power plants, challenges, applications, and blockchain integration." *Discover Applied Sciences* 7.4 (2025): 1-22.
- [12] Sarmah, Simanta Shekhar. "Understanding blockchain technology." *Computer science and engineering* 8.2 (2018): 23-29.
- [13] Zhang, Rui, Rui Xue, and Ling Liu. "Security and privacy on blockchain." *ACM Computing Surveys (CSUR)* 52.3 (2019): 1-34.
- [14] Zheng, Zibin, et al. "An overview on smart contracts: Challenges, advances and platforms." *Future Generation Computer Systems* 105 (2020): 475-491.
- [15] Münsing, Eric, Jonathan Mather, and Scott Moura. "Blockchains for decentralized optimization of energy resources in microgrid networks." *2017 IEEE conference on control technology and applications (CCTA)*. IEEE, 2017.
- [16] AlSkaif, Tarek, and Gijs Van Leeuwen. "Decentralized optimal power flow in distribution networks using blockchain." *2019 International Conference on Smart Energy Systems and Technologies (SEST)*. IEEE, 2019.

- [17] Vardakas, John S., Nizar Zorba, and Christos V. Verikoukis. "A survey on demand response programs in smart grids: Pricing methods and optimization algorithms." *IEEE Communications Surveys & Tutorials* 17.1 (2014): 152-178.
- [18] Jindal, Anish, Mukesh Singh, and Neeraj Kumar. "Consumption-aware data analytical demand response scheme for peak load reduction in smart grid." *IEEE Transactions on Industrial Electronics* 65.11 (2018): 8993-9004.
- [19] Rasheed, Muhammad Babar, and María D. R-Moreno. "Minimizing pricing policies based on user load profiles and residential demand responses in smart grids." *Applied Energy* 310 (2022): 118492.
- [20] Mitrea, Dan, et al. "Evolutionary game for incentivizing social cooperation of prosumers in transactive energy communities." *Energy and Buildings* 327 (2025): 115057.
- [21] Pop, Cristina Bianca, et al. "Review of bio-inspired optimization applications in renewable-powered smart grids: Emerging population-based metaheuristics." *Energy Reports* 8 (2022): 11769-11798.
- [22] Mureddu, Mario, Emilio Ghiani, and Fabrizio Pilo. "Smart grid optimization with blockchain based decentralized genetic algorithm." *2020 IEEE Power & Energy Society General Meeting (PESGM)*. IEEE, 2020.
- [23] Juneja, Mudita, and S. K. Nagar. "Particle swarm optimization algorithm and its parameters: A review." *2016 International conference on control, computing, communication and materials (ICCCCM)*. IEEE, 2016.
- [24] Shami, Tareq M., et al. "Particle swarm optimization: A comprehensive survey." *Ieee Access* 10 (2022): 10031-10061.
- [25] Gad, Ahmed G. "Particle swarm optimization algorithm and its applications: a systematic review." *Archives of computational methods in engineering* 29.5 (2022): 2531-2561.
- [26] A. Trindade. "ElectricityLoadDiagrams20112014," UCI Machine Learning Repository, 2015. [Online]. Available: <https://doi.org/10.24432/C58C86>.
- [27] Shi, Kun, et al. "Smart community energy cost optimization taking user comfort level and renewable energy consumption rate into consideration." *Processes* 7.2 (2019): 63.
- [28] Giordano, Andrea, Carlo Mastroianni, and Luigi Scarcello. "Optimization model for IoT-aware energy exchange in energy communities for residential users." *Electronics* 9.6 (2020): 1003.
- [29] Lazzari, Florencia, et al. "Optimizing planning and operation of renewable energy communities with genetic algorithms." *Applied energy* 338 (2023): 120906.
- [30] Hamze, Fouad Jose Salazar, and Ricardo Guerrero-Lemus. "Development and evaluation of an optimization model for enhancing energy communities sharing coefficients in Spain." *Energy and Buildings* (2025): 115951.